

Graph traversal

Przemysław Gordinowicz

Institute of Mathematics,
Łódź University of Technology



Graphs & Social Networks
Lecture 5

Outline



- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games

Outline



- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games



Reminder

Definition

Let G be a graph, and G_0 its subgraph. We say that G_0 is **spanning subgraph** of G , when $V(G) = V(G_0)$.

Definition

A **spanning tree** is a spanning subgraph that is a tree.

Observation (of „railway builders”)

Every connected graph has a spanning tree.

Natural question



Question

Given graph G — how to find its spanning tree?

Natural question



Question

Given graph G — how to find its spanning tree?

Maybe a tree rooted at given vertex v ?

Maybe one can demand some additional properties of this tree?

How to safely explore the maze?



N. Bambini — Ariadne and Theseus

Outline



- 1 Introduction
- 2 General schema of graph traversal**
- 3 Deep-first search
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

 choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T$;

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

 choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T$;

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$

Notation $(V_T, E_T) + (w, uw) := (V_T \cup \{w\}, E_T \cup \{uw\}).$



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T;$

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$

Which edge to choose — it is decided by the specific method.



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

 choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T;$

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$

Theorem

It works!

The proof is simple.



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T;$

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$

- “deep-first search” — u added to T as late as possible
- “breadth-first search” — u added to T as early as possible



General schema of graph traversal

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Traverse(V, E, v)

Function Tree(G, v);

$V_T := \{v\};$

$E_T := \emptyset;$

while $\exists uw \in E \ u \in V_T \wedge w \in V \setminus V_T$ **do**

choose an edge $uw \in E$ such that $u \in V_T \wedge w \in V \setminus V_T;$

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

Tree := $(V_T, E_T);$

- “deep-first search” — u farthest from v (distance in T)
- “breadth-first search” — u closest to v (distance in T) (?)

Outline



- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search**
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games



Deep-first search (DFS)

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Deep-first search(V, E, v)

```
Procedure DFS( $u$ );  
  for each  $w \in N(u)$  do  
    if  $w \notin V_T$  then  
       $(V_T, E_T) := (V_T, E_T) + (w, uw)$ ;  
      DFS( $w$ );
```

```
 $V_T := \{v\}$ ;  
 $E_T := \emptyset$ ;  
DFS( $v$ );
```



Deep-first search — algorithm analysis

- The algorithm is correct
- When a vertex is added to the tree, it is the farthest from the root of all the possible vertices that can be added
- The ends of each edge of a connected component are in a descendant–ancestor relation in the (rooted) DFS tree
- The computational complexity is $O(|V| + |E|)$ for the graph represented by an adjacency list
- The computational complexity is $O(|V|^2)$ for the graph represented by an adjacency matrix



Tree traversal — vertex ordering

Depth-first search of rooted trees, starting from the root, allows you to create a linear order $\leq$ of vertices (the order in which they will be listed) depending on the shape of the tree (order $\preceq$) and a specific order of the children ($\triangleleft$) of each vertex:

- **preorder** — first we list a given vertex, then we traverse the subtree rooted in it
- **postorder** — first we traverse the subtree rooted a given vertex, then we list the vertex
- **inorder** — for binary trees only, this is a natural order of BST — first we traverse a subtree rooted in the left child of a given vertex, then we list the vertex, and traverse a subtree rooted in the right child.

Outline



- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search
- 4 Breadth-first search**
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games



Breadth first search (BFS)

Input: Graph $G = (V, E)$ and a vertex $v \in V$.

Result: Tree $T = (V_T, E_T)$ spanning connected component of G that contains v .

Procedure BFS(V, E, v);

$V_T := \{v\};$

$E_T := \emptyset;$

$Queue \leftarrow v;$

while $Queue \neq \emptyset$ **do**

$u \leftarrow Queue;$

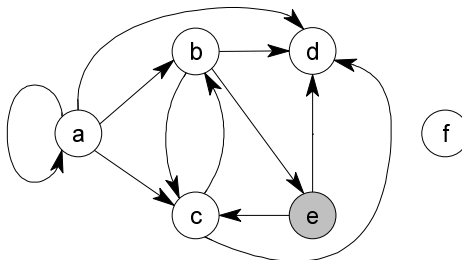
for each $w \in N(u)$ **do**

if $w \notin V_T$ **then**

$(V_T, E_T) := (V_T, E_T) + (w, uw);$

$Queue \leftarrow w;$

An example for digraphs

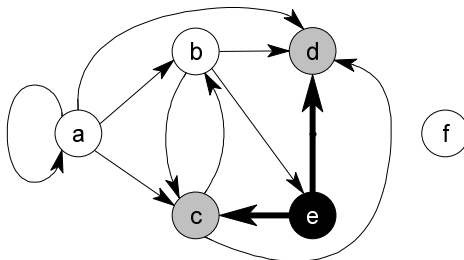


	a	b	c	d	e	f
<i>Visited</i>	0	0	0	0	1	0
<i>Previous</i>	0	0	0	0	0	0
<i>Distance</i>	∞	∞	∞	∞	0	∞

Queue e

Remark: bold edges do not create a tree — bold shows processed edges.

An example for digraphs

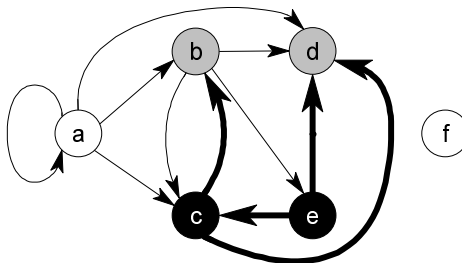


	a	b	c	d	e	f
<i>Visited</i>	0	0	1	1	1	0
<i>Previous</i>	0	0	e	e	0	0
<i>Distance</i>	∞	∞	1	1	0	∞

Queue c d

Remark: bold edges do not create a tree — bold shows processed edges.

An example for digraphs

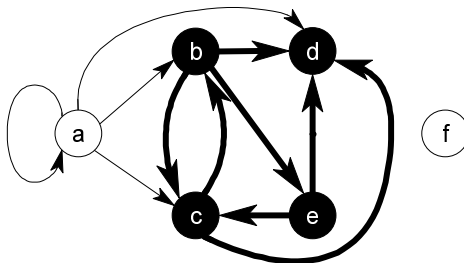


	a	b	c	d	e	f
<i>Visited</i>	0	1	1	1	1	0
<i>Previous</i>	0	c	e	e	0	0
<i>Distance</i>	∞	2	1	1	0	∞

Queue d b

Remark: bold edges do not create a tree — bold shows processed edges.

An example for digraphs



	a	b	c	d	e	f
<i>Visited</i>	0	1	1	1	1	0
<i>Previous</i>	0	c	e	e	0	0
<i>Distance</i>	∞	2	1	1	0	∞

Queue

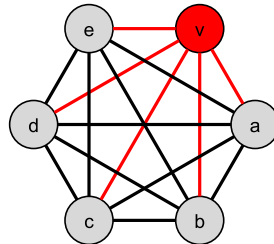
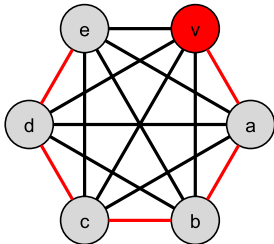
Remark: bold edges do not create a tree — bold shows processed edges.

Breadth-first search — algorithm analysis

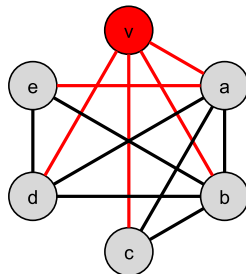
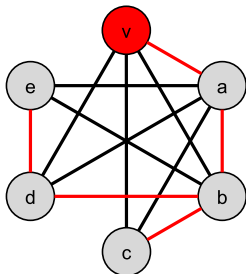
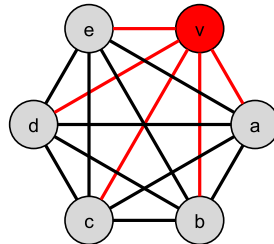
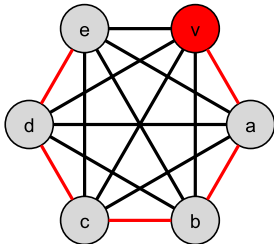


- The algorithm is correct
- When a vertex is added to the tree, it is the closest to the root of all the possible vertices that can be added
- The resulting spanning tree contains the shortest paths from vertex v to all vertices reachable from v .
- The computational complexity is $O(|V| + |E|)$ for the graph represented by an adjacency list
- The computational complexity is $O(|V|^2)$ for the graph represented by an adjacency matrix

Deep-first vs breadth-first search — differences



Deep-first vs breadth-first search — differences



Outline



- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications**
- 6 Digression — graph searching games

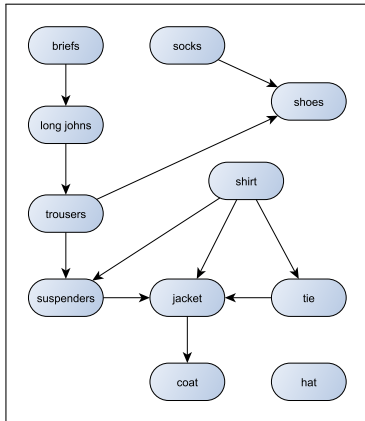
Generalised deep-first search of digraphs



```
Procedure DFS( $u$ );  
   $visited(u) = |V_F|$ ;  
  for each  $w \in N^+(u)$  do  
    if  $w \notin V_T$  then  
       $(V_F, A_F) := (V_F, A_F) + (w, uw)$ ;  
      DFS( $w$ );  
   $processed(v) = |V_F|$ ;
```

```
Procedure GDFS( $G$ );  
   $V_F := \emptyset$ ;  $A_F := \emptyset$ ;  
  for each  $v \in V(G)$  do  
     $visited(v) := processed(v) := 0$ ;  
  for each  $v \in V(G)$  do  
    if  $v \notin V_F$  then  
       $V_F := V_F \cup \{v\}$ ;  
      DFS( $v$ );
```

How to dress an (indecisive) computer scientist?

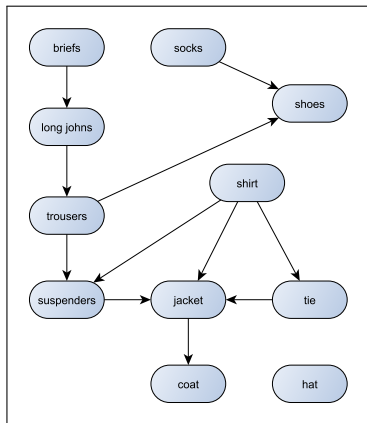


cf. Cormen et al., 2009

Problem

Given the order of tasks, schedule them.

How to dress an (indecisive) computer scientist?



cf. Cormen et al., 2009

Problem

Given dag $G = (V, A)$
determine a total order $\leq$ on
the vertex set such that for
any $u, v \in V$ one holds
 $(u, v) \in A \Rightarrow u \leq v$.

In other words the task is to
find a linear extension of par-
tial order being a transitive clo-
sure of the relation A .



Topological sorting

Problem

Given acyclic digraph $G = (V, A)$ determine a total order $\leq$ on the vertex set such that for any $u, v \in V$ one holds $(u, v) \in A \Rightarrow u \leq v$.

Topological sorting of digraph G

$\text{GDFS}(G);$

for each $v \in V(G)$ **do**

$\text{order}(v) := |V(G)| * \text{processed}(v) - \text{visited}(v);$

One holds $u \leq v \Leftrightarrow \text{order}(u) \geq \text{order}(v)$.



Topological sorting

Problem

Given acyclic digraph $G = (V, A)$ determine a total order $\leq$ on the vertex set such that for any $u, v \in V$ one holds $(u, v) \in A \Rightarrow u \leq v$.

Topological sorting of digraph G

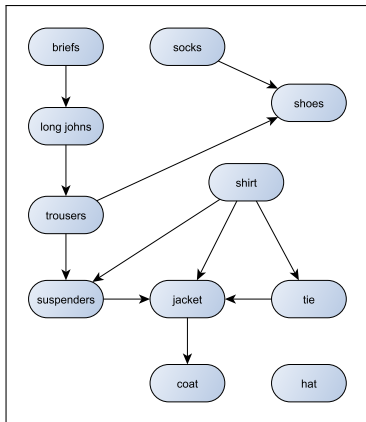
$\text{GDFS}(G);$

for each $v \in V(G)$ **do**

$\text{order}(v) := |V(G)| * \text{processed}(v) - \text{visited}(v);$

One holds $u \leq v \Leftrightarrow \text{order}(u) \geq \text{order}(v)$.

Topological sorting — an example



By running GDFS and processing the vertices in alphabetical order we get:

vertex	visited	processed	order
briefs	1	7	76
coat	7	7	70
jacket	6	7	71
long johns	2	7	75
hat	8	8	80
shirt	9	10	101
shoes	4	4	40
socks	11	11	110
suspenders	5	7	72
tie	10	10	100
trousers	3	7	74

Hence, the proper order of dressing is: socks, shirt, tie, hat, briefs, long johns, trousers, suspenders, jacket, coat, shoes.

Exercise

What dressing order will we get if GDFS is running in reverse alphabetical order?



Strongly connected components

The presented topological sorting algorithm may be run also on digraph with cycles. Of course, then the vertex ordering will not extend the relation defined by digraph arcs. However, acyclicity of the digraph can be checked.

```
GDFS( $G$ );  
for each  $v \in V(G)$  do  
     $order(v) := |V(G)| * processed(v) - visited(v)$ ;  
Sort vertices descending by order;  
GDFS( $G^T$ );
```

Reminder: $G^T = (V, A^T)$ denotes the opposite digraph of $G = (V, A)$, that is
 $A^T = \{(u, v) : (v, u) \in A\}$.

When running GDFS(G^T), the obtained vertex order should be used in the main loop.

Strongly connected components structure of digraph.



Strongly connected components

The presented topological sorting algorithm may be run also on digraph with cycles. Of course, then the vertex ordering will not extend the relation defined by digraph arcs. However, acyclicity of the digraph can be checked.

```
GDFS( $G$ );  
for each  $v \in V(G)$  do  
     $order(v) := |V(G)| * processed(v) - visited(v)$ ;  
Sort vertices descending by order;  
GDFS( $G^T$ );
```

Reminder: $G^T = (V, A^T)$ denotes the opposite digraph of $G = (V, A)$, that is
 $A^T = \{(u, v) : (v, u) \in A\}$.

When running $GDFS(G^T)$, the obtained vertex order should be used in the main loop.

Strongly connected components structure of digraph.

Applications of breadth-first search



- 1 By simple add-inn (condition checking) one can identify acyclic or bipartite graphs (DFS also allows it).
- 2 Logical games (choosing the optimal move within a limited time frame).
- 3 Many more we will see later, as this is important tool used in many advanced graph algorithms.

Outline



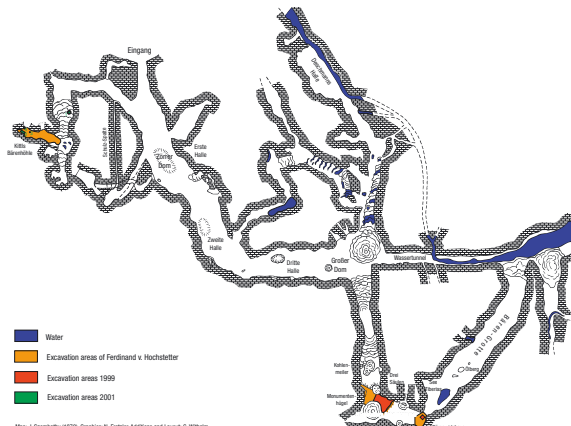
- 1 Introduction
- 2 General schema of graph traversal
- 3 Deep-first search
- 4 Breadth-first search
- 5 Deep-first and breadth-first search applications
- 6 Digression — graph searching games**

Inspiration



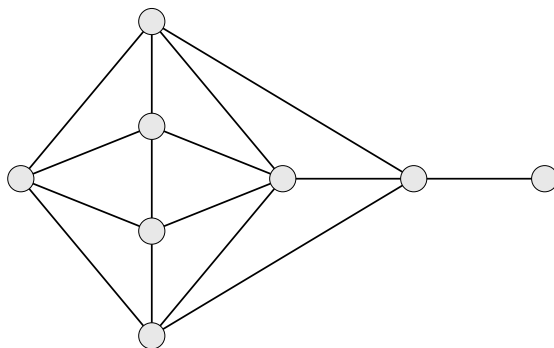
R.L. Breisch, An intuitive approach to speleotopology,
Southwestern Cavers 6 (1967), 72–78.

Graph searching vs graph traversal



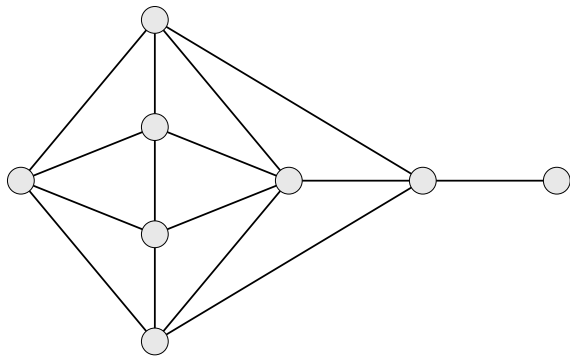
R.L. Breisch, An intuitive approach to speleotopology,
Southwestern Cavers 6 (1967), 72–78.

Graph searching vs graph traversal



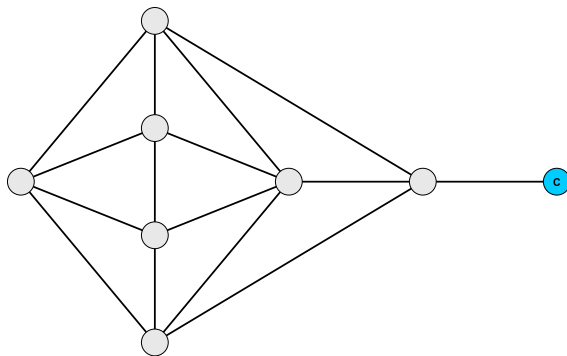
T.D. Parsons, Pursuit-evasion in a graph, *Theory and applications of graphs, Lecture Notes in Math.*, **642** (1978) 426–441.

Cop and robber game



Quilliot [1978], Nowakowski, Winkler [1983].

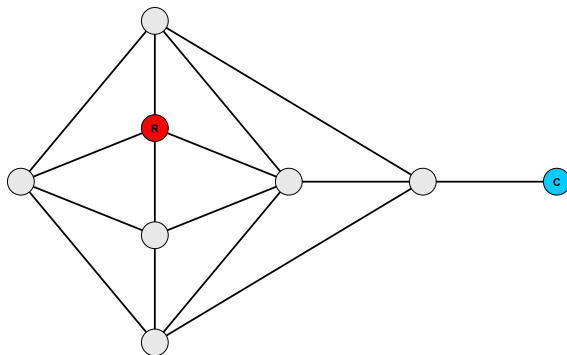
Cop and robber game



At first players put their pieces on selected vertices of a graph — the first player put a **cop**, the second one — a **robber**.

Quilliot [1978], Nowakowski, Winkler [1983].

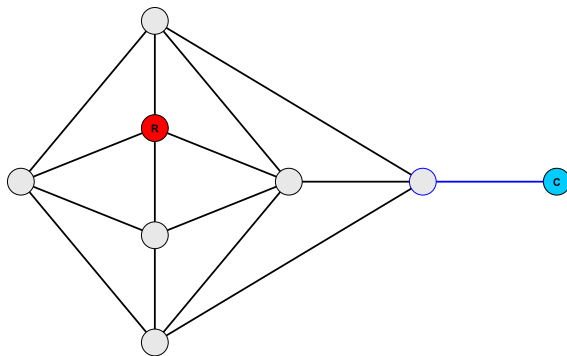
Cop and robber game



At first players put their pieces on selected vertices of a graph — the first player put a **cop**, the second one — a **robber**.

Quilliot [1978], Nowakowski, Winkler [1983].

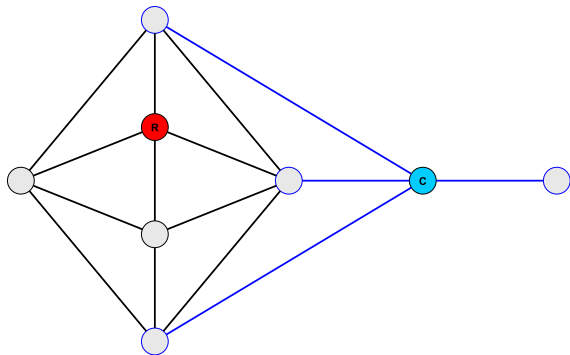
Cop and robber game



Then they move their pieces alternately at distance at most 1.

Quilliot [1978], Nowakowski, Winkler [1983].

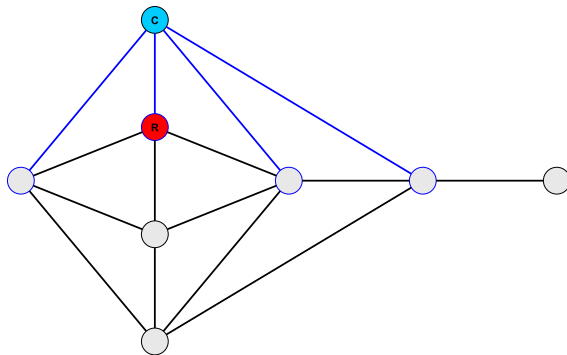
Cop and robber game



Then they move their pieces alternately at distance at most 1.

Quilliot [1978], Nowakowski, Winkler [1983].

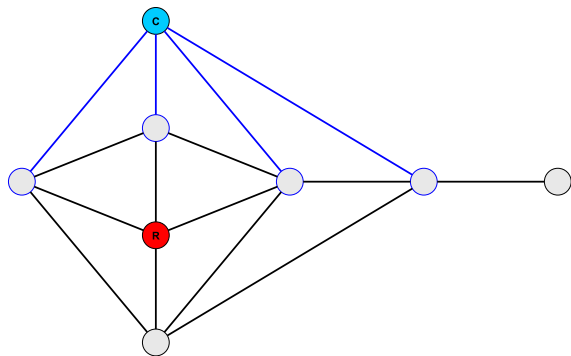
Cop and robber game



Then they move their pieces alternately at distance at most 1.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game

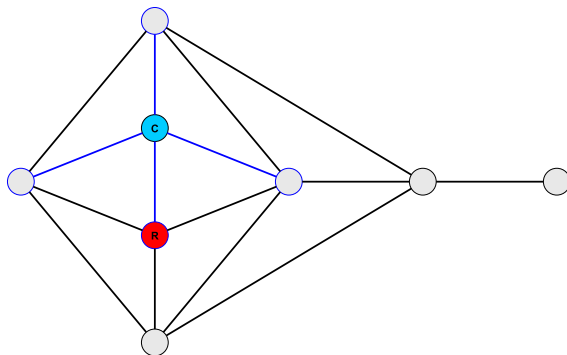


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game

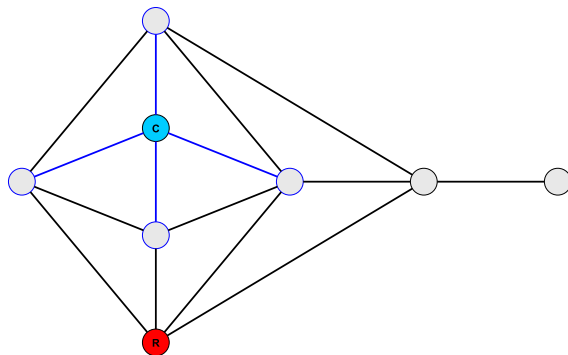


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game

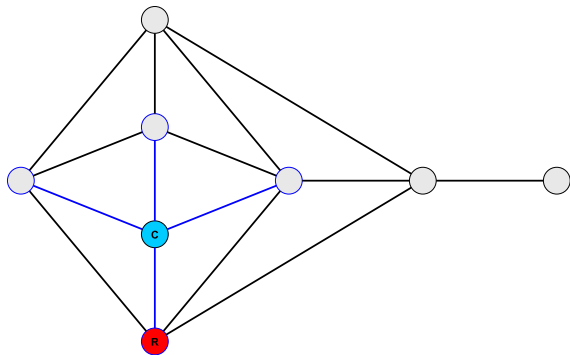


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game



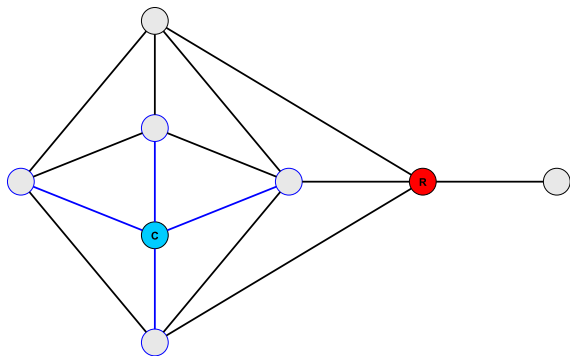
Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].



Cop and robber game

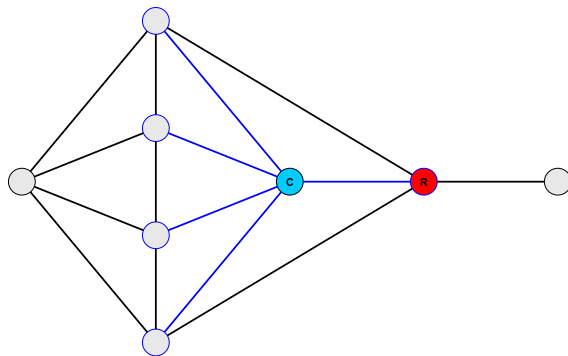


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game



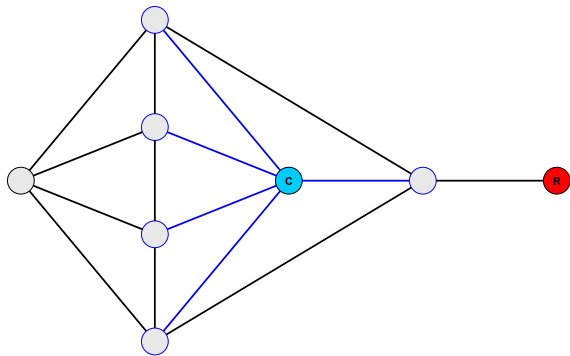
Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].



Cop and robber game



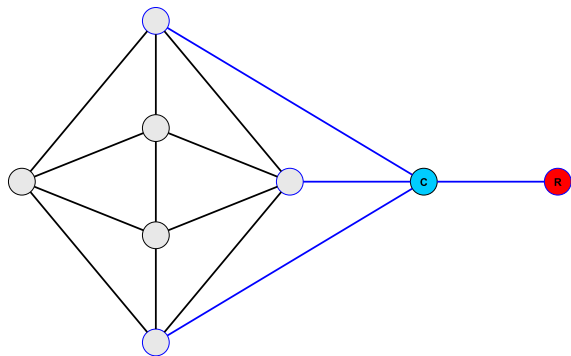
Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].



Cop and robber game

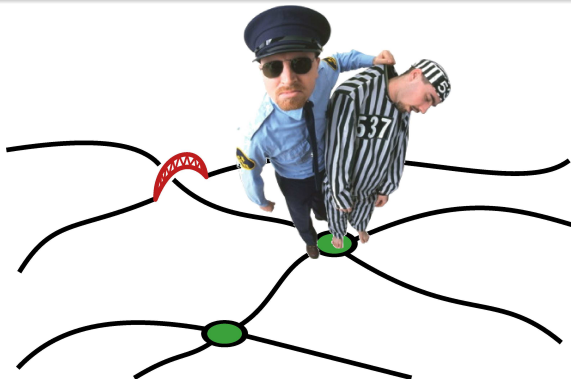


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game

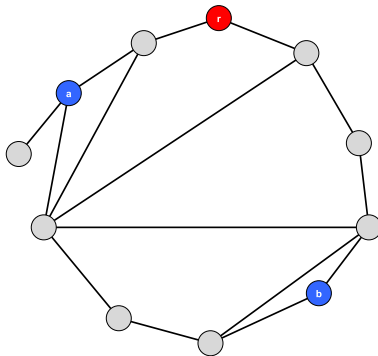


Then they move their pieces alternately at distance at most 1.

The goal for the cop is to catch the robber. For the robber — not to get caught within a finite time.

Quilliot [1978], Nowakowski, Winkler [1983].

Cop and robber game — many cops

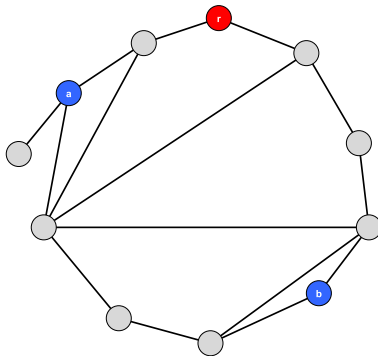


One cop is not enough for most of the graphs.

One may analyse a *cop number* of graph G , i.e. the smallest $k \in \mathbb{N}^+$ such that k cops have a winning strategy for the game played on the graph G .

Quilliot [1978], Nowakowski, Winkler [1983] Aigner, Fromme [1984].

Cop and robber game



One cop is not enough for most of the graphs.

One may analyse a *cop number* of graph G , i.e. the smallest $k \in \mathbb{N}^+$ such that k cops have a winning strategy for the game played on the graph G .

Quilliot [1978], Nowakowski, Winkler [1983] Aigner, Fromme [1984].

But also (incl.) Bonato, G., Kinnersley, Prałat [2013], Dudek, G., Prałat [2014], Bonato, G., Hahn [2017], Gavenčiak, G., Jelínek, Klavík, Kratochvíl [2018].